Software Anti-Patterns Detection

Iyashi Pal

1. Introduction

Fault prediction plays a crucial role in the software development life cycle, as it helps identify fault-prone modules early and reduces the probability of software failure. By detecting design flaws (anti-patterns), developers can prioritize maintenance efforts and apply targeted verification and validation strategies to improve overall software quality.

In this assignment, the commercial static analysis tool inFusion was used to detect design flaws in the Java project Apache Commons Lang across five consecutive versions:

- commons-lang-3.16.0
- commons-lang-3.17.0
- commons-lang-3.18.0
- commons-lang-3.19.0
- commons-lang-3.20.0

The goal of this study is to analyze how quality deficit indices (complexity, coupling, cohesion, encapsulation, inheritance) evolve as the software evolves and to determine which design quality is most in deficit.

---

2. Tool Description – inFusion

inFusion is a commercial static analysis tool designed to detect software anti-patterns (code smells and design flaws). Each detected flaw is associated with a numerical severity score, representing its relative impact on overall system quality.

The tool detects several types of anti-patterns, including:

- Blob Class
- Blob Operations
- Data Clumps
- Data Class
- Cyclic Dependencies
- SAP Breakers
- Internal Duplication
- Schizophrenic Class
- Traditional Breakers
- Unstable Dependencies

inFusion also computes **quality deficit indices**, which aggregate detected flaws into five main quality dimensions:

- Complexity
- Coupling
- Cohesion

- Encapsulation
- Inheritance

These indices allow quantitative comparison across software versions.

---

3. Methodology

3.1 Project Selection

Apache Commons Lang was selected because:

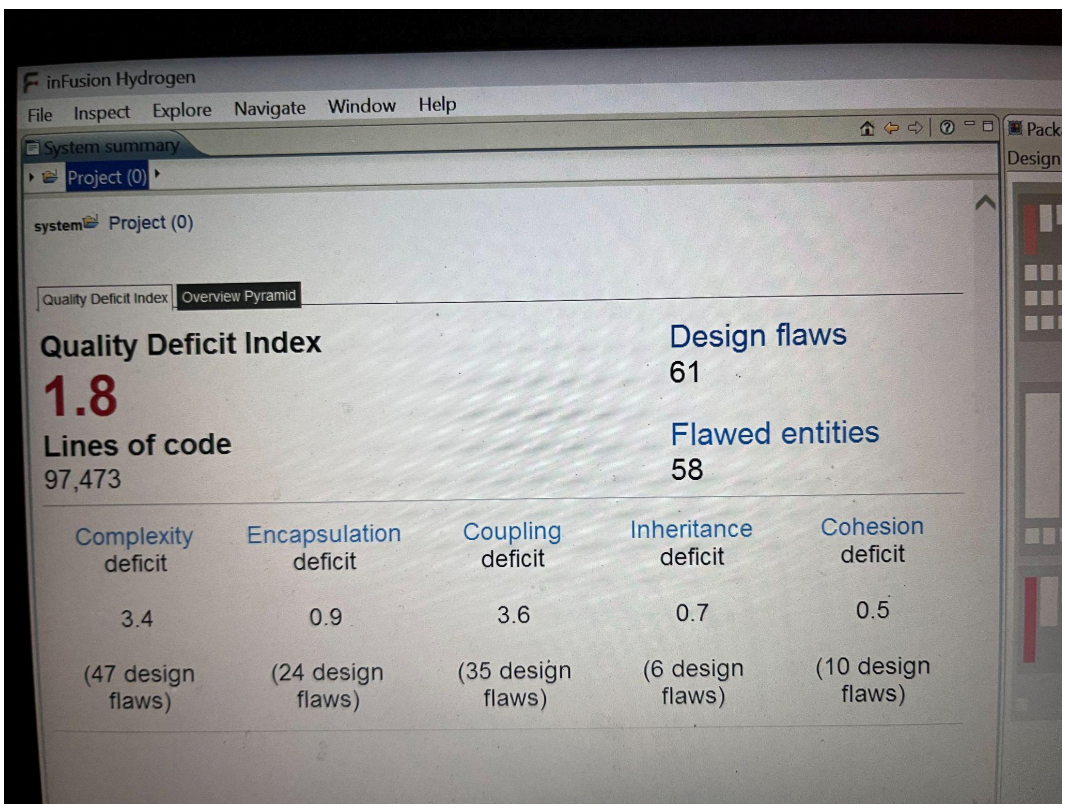
- It is a Java-based open-source project.
- It has multiple stable releases.
- Its size is within the small/medium project constraint (≤ 100,000 LOC).
- It was previously analyzed in Assignment 01, allowing consistency of study.

Five consecutive releases (3.16–3.20) were selected to observe quality evolution.

3.2 Data Collection

For each version:

1. The source code was imported into inFusion.
2. The tool generated reports of detected design flaws.



3. The numerical values for each quality dimension were exported to Excel.
4. Both raw flaw counts and quality deficit indices were recorded.

	A	B	C	D	E	F	G
1	Version	Complexity deficit	Encapsulation deficit	Coupling deficit	Inheritance deficit	Cohesion deficit	Quality Deficit Index
2	3.16	3.4	0.9	3.6	0.7	0.5	1.8
3	3.17	3.4	0.9	3.6	0.7	0.5	1.8
4	3.18	3	0.8	3	0.4	0.5	1.5
5	3.19	2.9	0.8	2.7	0.4	0.5	1.4
6	3.2	2.9	0.9	2.7	0.4	0.5	1.5

	A	B	C	D	E	F	G
1	Version	Complexity design flaws	Encapsulation design flaws	Coupling design flaws	Inheritance design flaws	Cohesion design flaws	Total design flaws
2	3.16	47	24	35	6	10	61
3	3.17	47	24	35	6	10	61
4	3.18	46	23	34	5	9	60
5	3.19	45	23	32	5	9	58
6	3.2	45	24	33	5	10	59
7							

The collected metrics include:

- Complexity design flaws
- Coupling design flaws
- Cohesion design flaws
- Encapsulation design flaws
- Inheritance design flaws
- Total design flaws
- Quality Deficit Index

3.3 Data Visualization

Using Excel, graphs were generated to visualize:

- Design flaw counts per quality dimension
- Quality deficit indices per dimension
- Overall Quality Deficit Index trend
- Total design flaws evolution

These visualizations allow analysis of how design quality changes across versions.

---

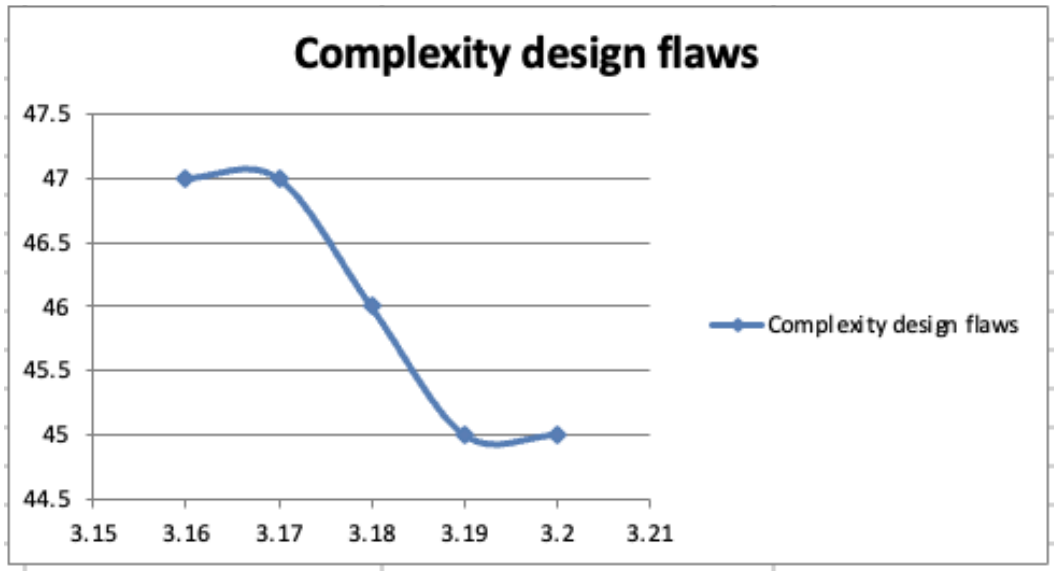
4. Results – Evolution of Design Flaws

4.1 Evolution of Design Flaws (Raw Counts)

Complexity Design Flaws

- 3.16: 47
- 3.17: 47
- 3.18: 46
- 3.19: 45
- 3.20: 45

Trend: Slight decreasing trend over time.

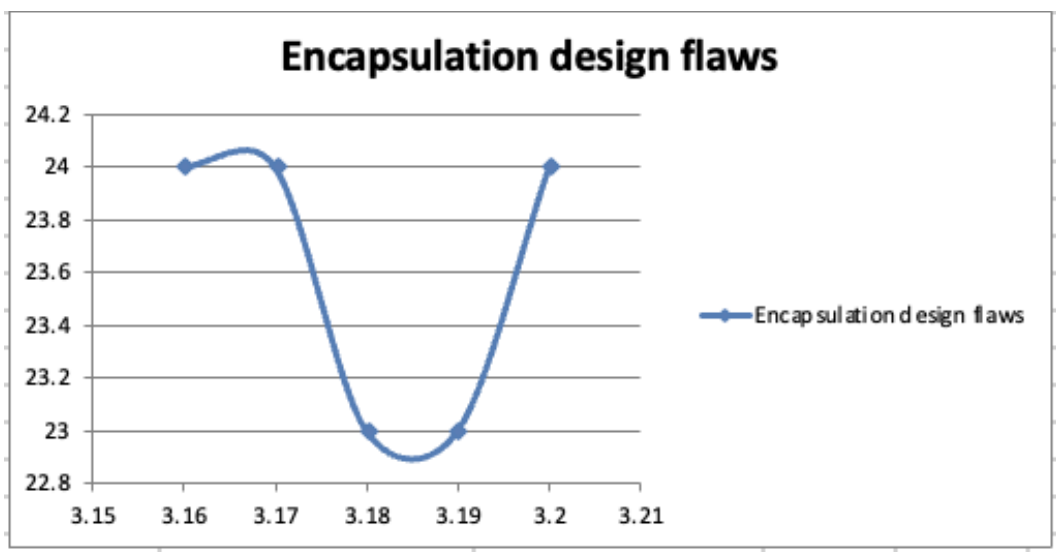


Interpretation: Although complexity flaws are the highest among all categories, there is a gradual improvement from version 3.17 onward. This suggests refactoring efforts or structural improvements.

Encapsulation Design Flaws

- Stable around 23–24 across all versions.

Trend: Minor fluctuation but generally stable.

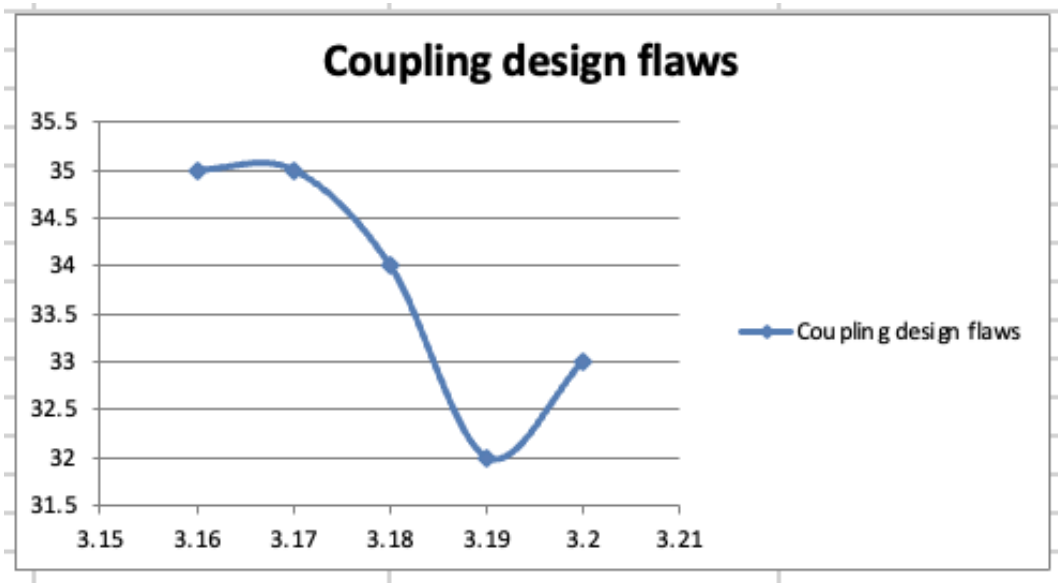


Interpretation: Encapsulation issues remain relatively controlled throughout the evolution.

Coupling Design Flaws

- 35 → 35 → 34 → 32 → 33

Trend: Clear decrease until version 3.19, slight increase in 3.20.

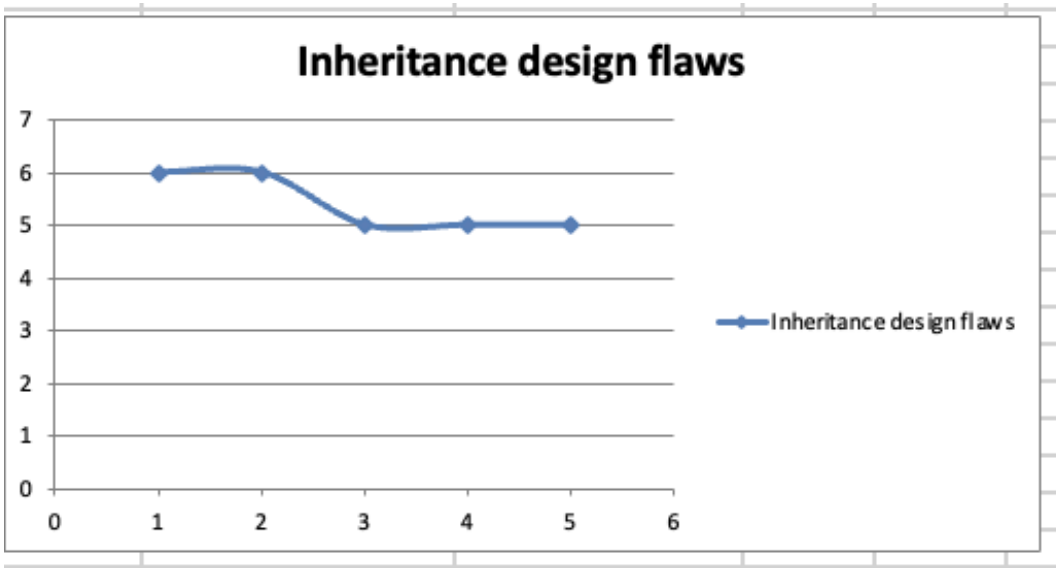


Interpretation: The project improved inter-class dependencies over time, reducing coupling problems, though a small regression appears in the last version.

Inheritance Design Flaws

- 6 → 6 → 5 → 5 → 5

Trend: Small but stable decrease.

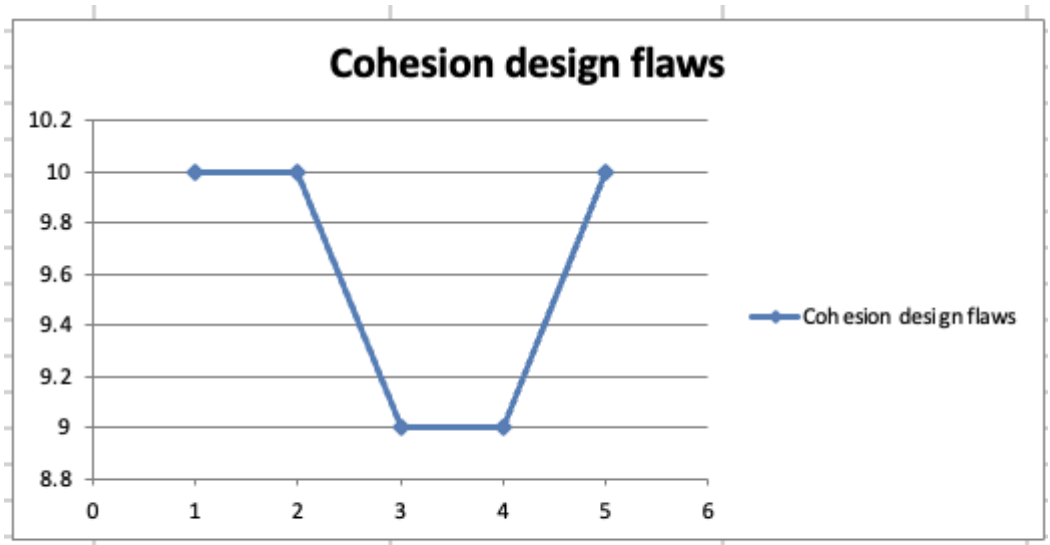


Interpretation: The inheritance structure remains relatively controlled with minimal deterioration.

Cohesion Design Flaws

- 10 → 10 → 9 → 9 → 10

Trend: Mostly stable.

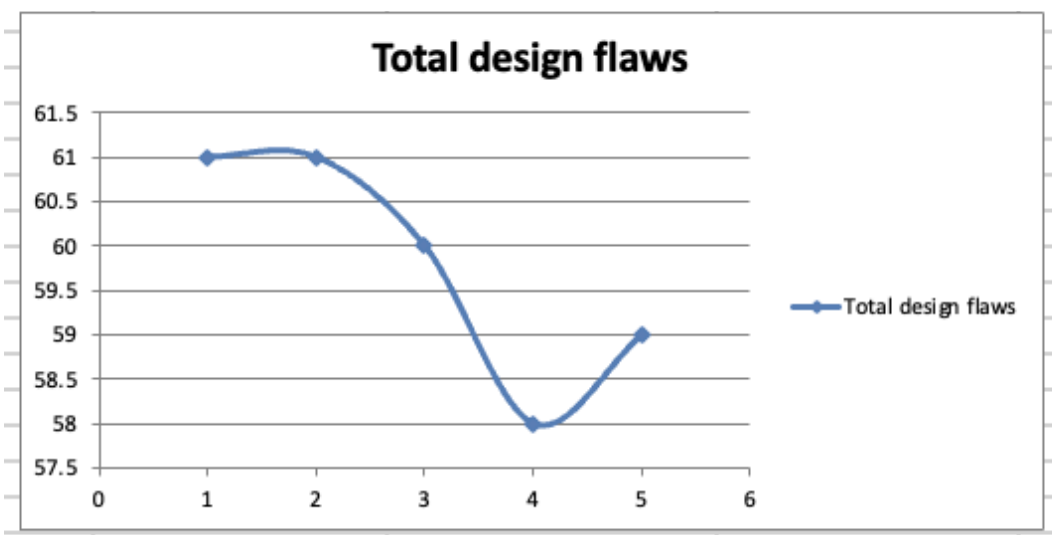


Interpretation: Cohesion issues are limited and do not significantly evolve across versions.

Total Design Flaws

- 61 → 61 → 60 → 58 → 59

Trend: Overall decreasing trend, slight increase in last version.



Interpretation: The overall structural quality improves over time with minor regression in version 3.20.

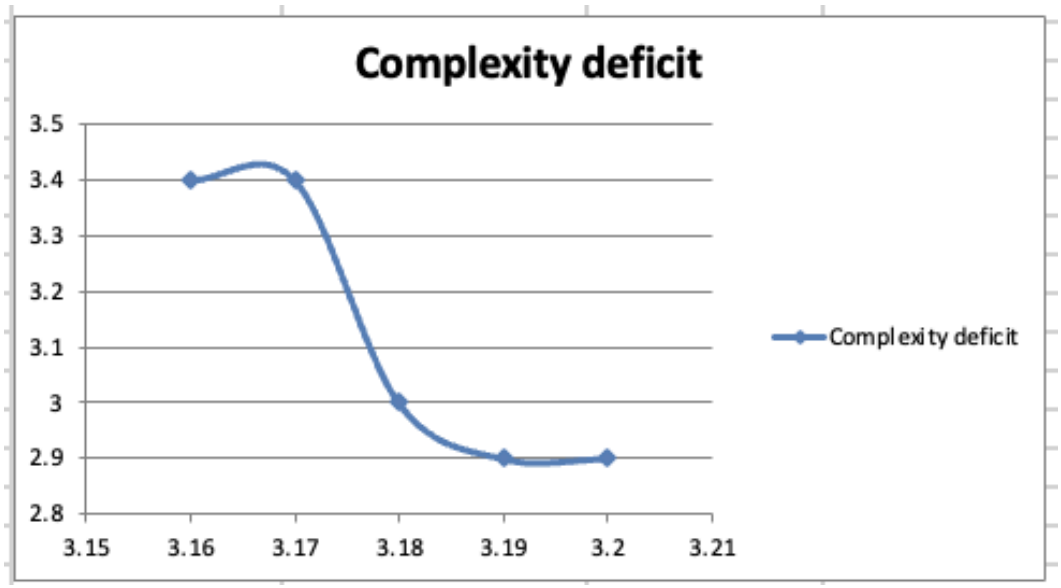
4.2 Quality Deficit Indices Analysis

Now focusing on normalized deficit values:

Complexity Deficit

- 3.4 → 3.4 → 3.0 → 2.9 → 2.9

Clear decreasing trend.

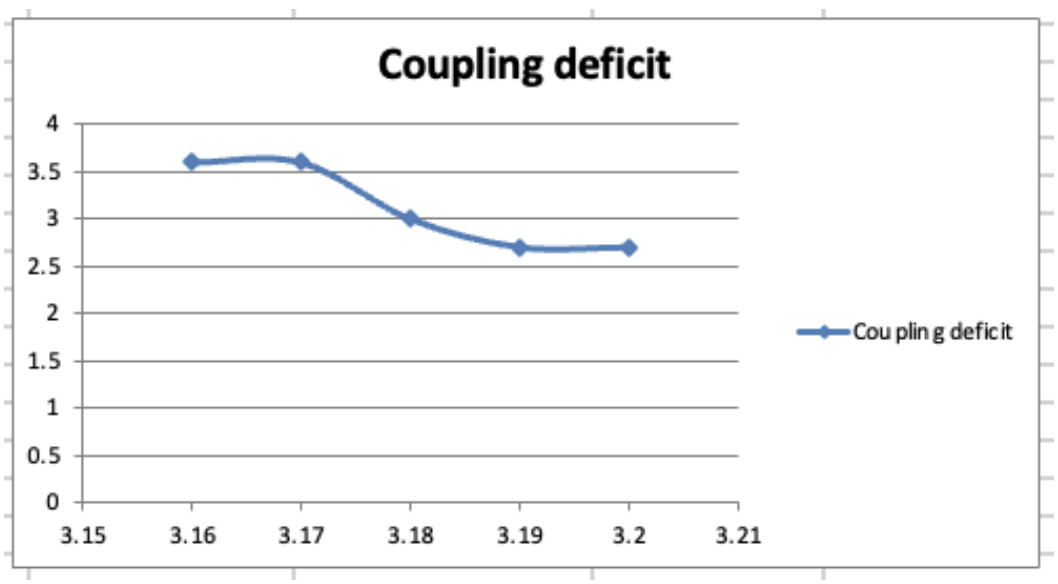


This indicates progressive improvement in structural complexity management.

Coupling Deficit

- 3.6 → 3.6 → 3.0 → 2.7 → 2.7

Strong decreasing trend.

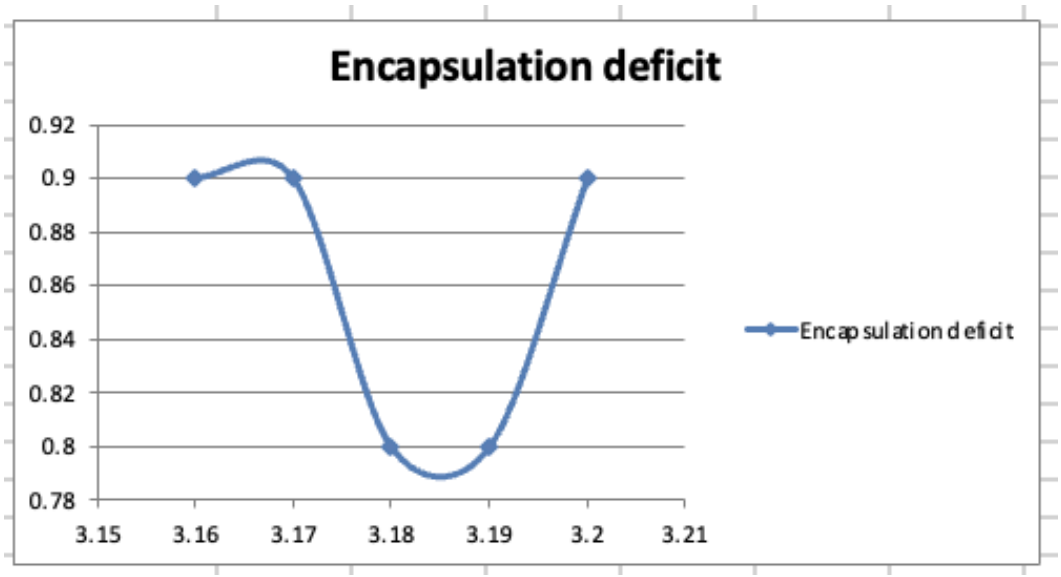


Coupling shows the most significant improvement among all dimensions.

Encapsulation Deficit

- 0.9 → 0.9 → 0.8 → 0.8 → 0.9

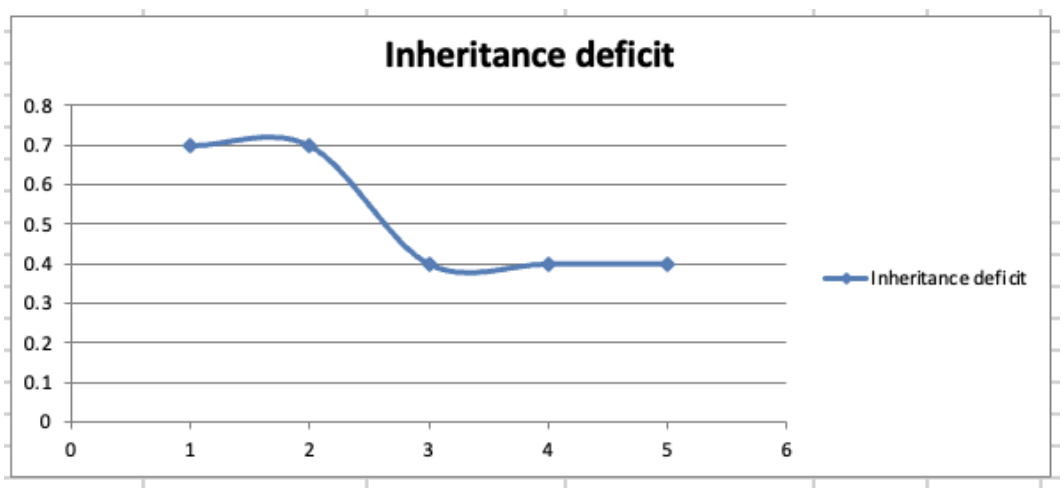
Very small variation.



Encapsulation remains relatively stable across releases.

Inheritance Deficit

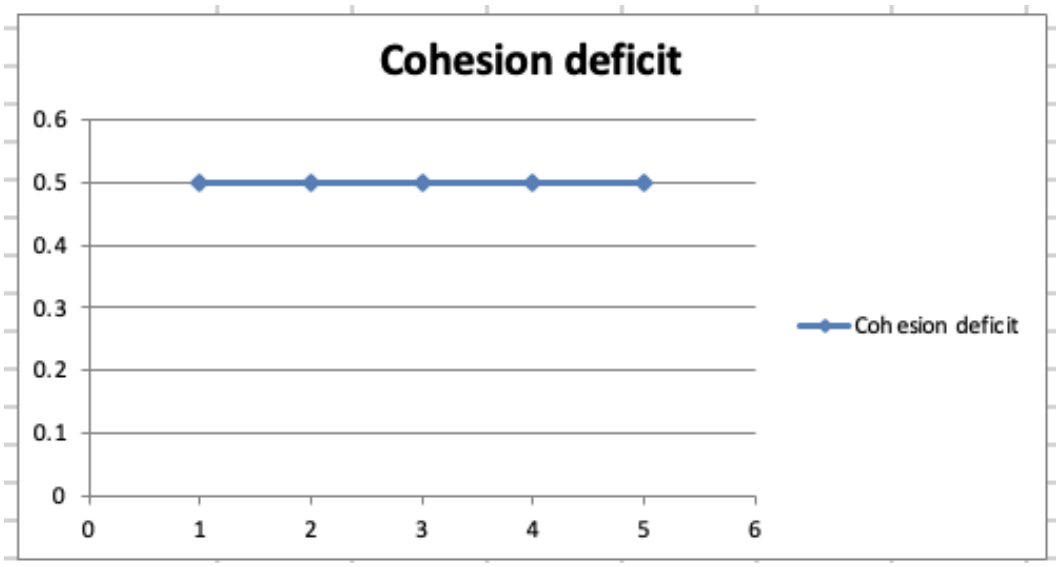
- 0.7 → 0.7 → 0.4 → 0.4 → 0.4



Noticeable improvement after version 3.17.

Cohesion Deficit

- Constant at 0.5 across all versions.



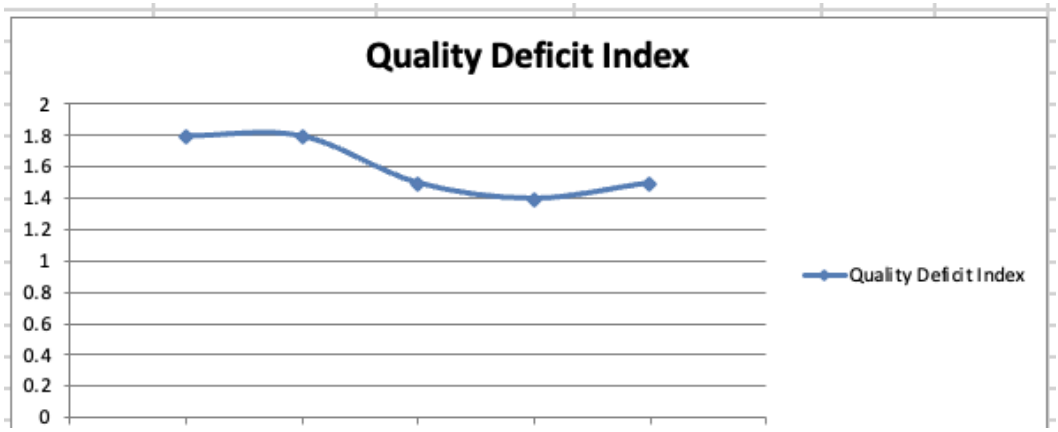
No change observed.

4.3 Quality Deficit Index (Overall)

- 1.8 → 1.8 → 1.5 → 1.4 → 1.5

Trend:

- Initial plateau
- Improvement in 3.18 and 3.19
- Slight regression in 3.20



Overall interpretation:

The system shows measurable improvement in structural quality between versions 3.17 and 3.19, indicating possible refactoring or architectural cleanup. The slight increase in 3.20 suggests additional functionality may have introduced minor structural complexity.

---

5. Identification of the Highest Quality Deficit

Across all versions, **Coupling Deficit and Complexity Deficit** consistently have the highest numerical values.

Among them, **Coupling Deficit is the highest overall**, particularly in earlier versions (3.6).

This indicates that excessive inter-class dependencies were the primary structural weakness of the system.

However, the steady reduction in coupling deficit demonstrates architectural improvement over time.

Therefore, coupling was the most critical design weakness, followed by complexity.

---

6. Relationship Between Size Increase and Design Flaws

Despite software evolution across five releases, the total number of design flaws slightly decreases rather than increases.

This suggests:

- Active maintenance and refactoring efforts
- Controlled growth of the codebase
- Architectural discipline

Instead of accumulating technical debt, the project demonstrates quality stabilization and even gradual improvement.

---

7. Experience with inFusion

Positives

- Clear classification of anti-patterns
- Quantitative scoring mechanism
- Easy comparison across versions
- Professional visualization of quality dimensions

Negatives

- Requires correct JRE/JDK configuration
- Commercial tool (limited accessibility)
- Initial setup can be complex

Overall Impression

inFusion provides a powerful quantitative approach to analyzing software design quality. The quality deficit indices are particularly useful for longitudinal analysis across software versions. It is highly suitable for academic research and professional quality assessment.

---

8. Conclusion

This study analyzed the evolution of design flaws in Apache Commons Lang across five versions using inFusion.

Key findings:

- Overall quality improved over time.
- Coupling and complexity were the most significant quality deficits.
- Refactoring efforts appear to have reduced architectural weaknesses.
- The system shows controlled evolution rather than technical debt accumulation.

The results demonstrate the importance of continuous structural monitoring and the effectiveness of automated anti-pattern detection tools.